



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student/s Name	1. ARIANNE BEVERLY MARQUEZ 2. LESTER ANTHONY RESPICIO
Section:	AN41
Professor:	MS. CRISOLA TAN
Date:	10/02/26

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)

- Google Drive for shared documentation
- GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
(Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.



Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{  
  "problem": {...},  
  "solution": {...}  
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.

Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

- ✓ Similarity score or ranking
- ✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

- ✓ "New case saved" or similar confirmation
- ✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system’s knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.

Student submits a math solution with repeated errors.

System retrieves past students with similar error patterns (e.g., misusing formulas).

It adapts feedback and personalized lessons based on successful interventions.

Learning:

System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks

```
import tkinter as tk
from tkinter import ttk

# TECHNICAL ASSESSMENT 3: CASE-BASED REASONING (CBR) SYSTEM
# Case Study 2: AI-Based Legal Decision Support System

def display_header():
    print(str('\nAI-BASED LEGAL DECISION SUPPORT SYSTEM
(CYBERCRIME & E-THEFT)\n'))
    print(str('\nProgrammed by: Arianne Beverly V. Marquez &
Lester Anthony G. Respicio\n'))

#STEP 3: Defining case base
case_base = [
    {
        'case_id': 'CR-2021-089',
        'title': 'People v. Arnaldo (Unauthorized ACH
Transfer)',
        'problem': {
            'offense_type': 'Unauthorized Bank Transfer',
            'transfer_medium': 'Web Banking Portal',
            'evidence_available': 'Server Access Logs',
            'disputed_amount': 150000.0,
            'primary_defense': 'Lack of Direct Attribution /
Shared Terminal'
        },
        'solution': {
```

```

        'precedent_title': 'G.R. No. 201142 - Doctrine of
Digital Chain of Custody',
        'applicable_law': 'Cybercrime Prevention Act &
E-Commerce Act (Sec. 33)',
        'core_argument': 'IP address mapping alone does not
establish physical identity beyond reasonable doubt without
session token correlation.',
        'recommended_motion': 'Motion to Suppress IP
Telemetry pending ISP subscriber identity verification.'
    }
},
{
    'case_id': 'CR-2022-144',
    'title': 'State v. Macatangay (Mobile Wallet Phishing)',
    'problem': {
        'offense_type': 'Digital Fraud & Identity Theft',
        'transfer_medium': 'Mobile Banking App',
        'evidence_available': 'SMS OTP Delivery Logs',
        'disputed_amount': 45000.0,
        'primary_defense': 'Compromised Credentials via SIM
Swap'
    },
    'solution': {
        'precedent_title': 'Crim. Case 4491 - Third-Party
Interception Standard',
        'applicable_law': 'Access Devices Regulation Act
(R.A. 8484)',
        'core_argument': 'Defendants credentials were
hijacked via fraudulent SIM duplication, negating criminal
intent.',
        'recommended_motion': 'Motion for Forensic
Examination of Telco SIM re-issuance audit trails.'
    }
},
{
    'case_id': 'CR-2023-305',
    'title': 'People v. Tan (API Scripted Transfer)',
    'problem': {
        'offense_type': 'Electronic Theft',
        'transfer_medium': 'Automated Clearing House API',
        'evidence_available': 'API Gateway Transaction
Payload',
        'disputed_amount': 520000.0,

```

```

        'primary_defense': 'Software Glitch / Involuntary
Beneficiary'
    },
    'solution': {
        'precedent_title': 'G.R. No. 198774 - Absence of
Animus Lucrandi',
        'applicable_law': 'Revised Penal Code Art. 308 in
rel. to R.A. 10175',
        'core_argument': 'Automated system race condition
caused erroneous double deposit without affirmative defendant
exfiltration.',
        'recommended_motion': 'Motion for Independent
Source-Code and Database Audit by court-appointed forensic
expert.'
    }
}
]

```

#STEP 4: Implementing similarity assessment

```

# Weighted scoring
FEATURE_WEIGHTS = {
    'offense_type': 0.30,
    'transfer_medium': 0.25,
    'evidence_available': 0.20,
    'primary_defense': 0.15,
    'amount_similarity': 0.10
}

# Simple Rule-Based Comparison
def calculate_amount_similarity(amt1, amt2):
    max_val = max(amt1, amt2)
    if max_val == 0:
        return 1.0
    return max(0.0, 1.0 - (abs(amt1 - amt2) / max_val))

# Attribute matching
def compute_case_similarity(new_case, past_case):
    p_new = new_case['problem']
    p_past = past_case['problem']

    score_offense = 1.0 if p_new['offense_type'].lower() ==
p_past['offense_type'].lower() else 0.0

```



```

        score_medium = 1.0 if p_new['transfer_medium'].lower() ==
p_past['transfer_medium'].lower() else 0.0
        score_evidence = 1.0 if p_new['evidence_available'].lower()
== p_past['evidence_available'].lower() else 0.3
        score_defense = 1.0 if p_new['primary_defense'].lower() ==
p_past['primary_defense'].lower() else 0.2
        score_amount =
calculate_amount_similarity(p_new['disputed_amount'],
p_past['disputed_amount'])

```

```

    global_similarity = (
        (score_offense * FEATURE_WEIGHTS['offense_type']) +
        (score_medium * FEATURE_WEIGHTS['transfer_medium']) +
        (score_evidence * FEATURE_WEIGHTS['evidence_available'])
+
        (score_defense * FEATURE_WEIGHTS['primary_defense']) +
        (score_amount * FEATURE_WEIGHTS['amount_similarity'])
    )
    return round(global_similarity, 4)

```

```

def retrieve_best_precedent(new_case, case_library):
    scored_cases = []
    for past_case in case_library:
        sim = compute_case_similarity(new_case, past_case)
        scored_cases.append((sim, past_case))

```

```

    scored_cases.sort(key=lambda item: item[0], reverse=True)
    return scored_cases[0], scored_cases

```

#STEP 5: Adapting retrieved solutions

```

def adapt_solution(retrieved_case, new_case):
    old_sol = retrieved_case['solution']
    new_prob = new_case['problem']

```

```

    adapted_argument = (
        f"{old_sol['core_argument']}In the current case, because
the transaction was executed via "
        f"{new_prob['transfer_medium']}, the defense asserts
that {new_prob['evidence_available']} "
        f"fails to conclusively prove intentional execution by
the client regarding the PHP {new_prob['disputed_amount']:.2f}
transfer."
    )

```

```

        adapted_motion = (
            f"Urgent Motion to present forensic counter-evidence
addressing {new_prob['evidence_available']} "
            f"and subpoena authentication records for
{new_prob['transfer_medium']} transactions pursuant to
{old_sol['applicable_law']}."
        )

    final_solution = {
        'applied_precedent': old_sol['precedent_title'],
        'applicable_law': old_sol['applicable_law'],
        'adapted_defense_argument': adapted_argument,
        'adapted_procedural_motion': adapted_motion
    }
    return final_solution

#STEP 6: Demonstrating learning (retain stage)
def retain_case(new_case, final_solution, case_library):
    retained_record = {
        'case_id': new_case['case_id'],
        'title': new_case.get('title', 'Adjudicated New
Incident'),
        'problem': new_case['problem'],
        'solution': final_solution
    }
    case_library.append(retained_record)
    return retained_record

# MAIN WORKFLOW (GUI Implementation)
def main():
    # Print the header to the terminal first
    display_header()

    root = tk.Tk()
    root.title('AI-Based Legal Decision Support System')
    root.geometry('900x750')
    root.configure(bg='#f0f0f0')

    # Ensure main container expands
    main_frame = tk.Frame(root, bg='#f0f0f0')
    main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

    # Centered Header (GUI)

```

```

        lbl_title = tk.Label(main_frame, text='AI-BASED LEGAL
DECISION SUPPORT SYSTEM', font=('Helvetica', 16, 'bold'),
bg='#f0f0f0')
        lbl_title.pack(pady=(0, 20))

        new_incident = {
            'case_id': 'CR-2026-901',
            'title': 'State v. S. Alvarez (E-Wallet Unauthorized
Transfer)',
            'problem': {
                'offense_type': 'Electronic Theft',
                'transfer_medium': 'Mobile Banking App',
                'evidence_available': 'SMS OTP Delivery Logs',
                'disputed_amount': 92000.0,
                'primary_defense': 'Compromised Credentials via SIM
Swap'
            }
        }

        # Left-aligned Table Title
        lbl_input = tk.Label(main_frame, text='[INPUT: INCOMING
CLIENT LEGAL ISSUE]', font=('Helvetica', 11, 'bold'),
bg='#f0f0f0')
        lbl_input.pack(anchor='w')

        # Treeview (Table)
        tree_frame = tk.Frame(main_frame)
        tree_frame.pack(fill=tk.X, pady=(5, 20))

        columns = ('Attribute', 'Details')
        tree = ttk.Treeview(tree_frame, columns=columns,
show='headings', height=6)
        tree.heading('Attribute', text='Case Attribute')
        tree.heading('Details', text='Client Details')

        # stretch=True allows the table to adapt when resizing
window
        tree.column('Attribute', width=200, anchor='w',
stretch=False)
        tree.column('Details', width=600, anchor='w', stretch=True)
        tree.pack(fill=tk.X, expand=True)

        tree.insert('', tk.END, values=('Case Reference',
new_incident['case_id']))

```

```

        tree.insert('', tk.END, values=('Allegation',
new_incident['problem']['offense_type']))
        tree.insert('', tk.END, values=('Transfer Medium',
new_incident['problem']['transfer_medium']))
        tree.insert('', tk.END, values=('Available Evidence',
new_incident['problem']['evidence_available']))
        tree.insert('', tk.END, values=('Disputed Amount', f"PHP
{new_incident['problem']['disputed_amount':, .2f}"))
        tree.insert('', tk.END, values=('Proposed Defense',
new_incident['problem']['primary_defense']))

```

```

# Scrollable Text Box for Logic Execution
text_frame = tk.Frame(main_frame)
text_frame.pack(fill=tk.BOTH, expand=True)

```

```

scrollbar = tk.Scrollbar(text_frame)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

```

```

text_output = tk.Text(text_frame, font=('Consolas', 10),
bg='#f8f9fa', wrap=tk.WORD, yscrollcommand=scrollbar.set)
text_output.pack(fill=tk.BOTH, expand=True)
scrollbar.config(command=text_output.yview)

```

```

def print_to_gui(msg):
    text_output.insert(tk.END, str(msg) + '\n')

```

```

# Integrating your exact string modifications with GUI
dashed lines
    print_to_gui('\nEXECUTING SIMILARITY ASSESSMENT
(RETRIEVAL)\n')

```

```

        (best_score, best_case), all_scores =
retrieve_best_precedent(new_incident, case_base)
        for score, item in all_scores:
            print_to_gui(f"-> Precedent [{item['case_id']}]
Similarity: {score * 100:.2f}%")

```

```

        print_to_gui(f"\n    !!! Best Match Found:
[{best_case['case_id']}] {best_case['title']} !!! \n")

```

```

    print_to_gui('\nEXECUTING ADAPTING RETRIEVED SOLUTION\n')

```

```

    adapted_strategy = adapt_solution(best_case, new_incident)

```

```

        print_to_gui(f"Applicable      Precedent      :
{adapted_strategy['applied_precedent']}")
        print_to_gui(f"\nAdapted      Legal      Argument:\n
{adapted_strategy['adapted_defense_argument']}")
        print_to_gui(f"\nRecommended      Actionable      Motion:\n
{adapted_strategy['adapted_procedural_motion']}\n")

    print_to_gui('\nEXECUTING RETAIN STAGE (LEARNING)\n')

    initial_count = len(case_base)
    new_record = retain_case(new_incident, adapted_strategy,
case_base)

    print_to_gui(f"Initial Case Base Count : {initial_count}")
    print_to_gui(f"Added Case ID
{new_record['case_id']}")
    print_to_gui(f"Updated Case Base Count : {len(case_base)}")

    text_output.config(state=tk.DISABLED)
    root.mainloop()

if __name__ == '__main__':
    main()

```

TECHNICAL 3 DOCUMENTATION

Chosen Case Study: **CASE STUDY 2: AI-Based Legal Decision Support System**

Case Base Description:

- This case base is represented with the use of structured Python dictionaries along with a primary list array. It initializes three historical legal precedents. Each case is divided into two main sub-dictionaries – **a problem object** where storing the offense type, transfer medium, available evidence, primary defense, and a continuous float for the disputed amount happen, **and solution object**, where storing the applied precedent title, governing law, core argument and recommended procedural motion occur.

Explanation of Similarity Method:

- The system evaluates relevance with the use of a hybrid weighted scoring model.

◆ **Attribute Matching**

- Categorical fields namely **offense_type**, **transfer_medium**, **evidence_available**, and **primary_defense** are all evaluated via string matching which yielded a 1.0 for a match and 0.0 (partial penalty) for a mismatch.

◆ **Weighted Scoring**

- The individual attribute scores are multiplied by a predefined priority dictionary (FEATURE_WEIGHTS), in order to prioritize legal classification (30%) and transfer medium (25%) over monetary differences (10%) to produce an output of a final global similarity percentage.

◆ **Simple-Rule Based Comparison**

- The continuous `disputed_amount` variable is evaluated using a normalized distance formula to find comparable financial damages without requiring exact matches.

$$1.0 - \frac{|amt1 - amt2|}{\max(amt1, amt2)}$$

Adaptation Logic

- The system reuses the **established statutory law and core argument from the most similar past case**, then uses **string interpolation (f-strings)** to dynamically revise the defense strategy. It explicitly **substitutes the retrieved precedent's context with the new problem's exact variables** specifically injecting the new `transfer_medium` (Mobile Banking App), `evidence_available` (SMS OTP Delivery Logs), and `disputed_amount` (₹ 92,000.00) in order to come up with a highly specific, trial-ready procedural motion for the client.

Program Execution Screenshots

```
PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Yani\AppData\Local\Programs\Python\Python39-64\python.exe -m pip install --upgrade pip
```

```
AI-BASED LEGAL DECISION SUPPORT SYSTEM (CYBERCRIME & E-THEFT)
```

```
Programmed by: Arianne Beverly V. Marquez & Lester Anthony G. Respicio
```

AI-Based Legal Decision Support System

AI-BASED LEGAL DECISION SUPPORT SYSTEM

[INPUT: INCOMING CLIENT LEGAL ISSUE]

Case Attribute	Client Details
Case Reference	CR-2026-901
Allegation	Electronic Theft
Transfer Medium	Mobile Banking App
Available Evidence	SMS OTP Delivery Logs
Disputed Amount	PHP 92,000.00
Proposed Defense	Compromised Credentials via SIM Swap

EXECUTING SIMILARITY ASSESSMENT (RETRIEVAL)

-> Precedent [CR-2022-144] Similarity: 64.89%

-> Precedent [CR-2023-305] Similarity: 48.77%

-> Precedent [CR-2021-089] Similarity: 15.13%

!!! Best Match Found: [CR-2022-144] State v. Macatangay (Mobile Wallet Phishing) !!!

EXECUTING ADAPTING RETRIEVED SOLUTION

Applicable Precedent : Crim. Case 4491 - Third-Party Interception Standard

Adapted Legal Argument:
Defendants credentials were hijacked via fraudulent SIM duplication, negating criminal intent. In the current case, because the transaction was executed via Mobile Banking App, the defense asserts that SMS OTP Delivery Logs fails to conclusively prove intentional execution by the client regarding the PHP 92,000.00 transfer.

Recommended Actionable Motion:
Urgent Motion to present forensic counter-evidence addressing SMS OTP Delivery Logs and subpoena authentication records for Mobile Banking App transactions pursuant to Access Devices Regulation Act (R.A. 8464).

EXECUTING RETAIN STAGE (LEARNING)

Initial Case Base Count : 3

Added Case ID : CR-2026-901

Updated Case Base Count : 4

AI-BASED LEGAL DECISION SUPPORT SYSTEM

[INPUT: INCOMING CLIENT LEGAL ISSUE]

Case Attribute	Client Details
Case Reference	CR-2026-901
Allegation	Electronic Theft
Transfer Medium	Mobile Banking App
Available Evidence	SMS OTP Delivery Logs
Disputed Amount	PHP 92,000.00
Proposed Defense	Compromised Credentials via SIM Swap

EXECUTING SIMILARITY ASSESSMENT (RETRIEVAL)

-> Precedent [CR-2022-144] Similarity: 64.89%
-> Precedent [CR-2023-305] Similarity: 40.77%
-> Precedent [CR-2021-089] Similarity: 15.13%

!!! Best Match Found: [CR-2022-144] State v. Macatangay (Mobile Wallet Phishing) !!!

EXECUTING ADAPTING RETRIEVED SOLUTION

Applicable Precedent : Crim. Case 4491 - Third-Party Interception Standard

Adapted Legal Argument:

Defendants credentials were hijacked via fraudulent SIM duplication, negating criminal intent. In the current case, because the transaction was executed via Mobile Banking App, the defense asserts that SMS OTP Delivery Logs fails to conclusively prove intentional execution by the client regarding the PHP 92,000.00 transfer.

Recommended Actionable Motion:

Urgent Motion to present forensic counter-evidence addressing SMS OTP Delivery Logs and subpoena authentication records for Mobile Banking App transactions pursuant to Access Devices Regulation Act (R.A. 8484).

EXECUTING RETAIN STAGE (LEARNING)

Initial Case Base Count : 3
Added Case ID : CR-2026-901
Updated Case Base Count : 4

TECHNICAL 3 REFLECTION

For this technical laboratory activity, the development of the Case-Based Reasoning system was done with the collaboration of Arianne and Lester in terms of orchestrating a complete 4R cycle featuring a dynamic graphical interface. Arianne focused on establishing the knowledge representation by structuring the Python case dictionaries and defining the mathematical logic for the weighted similarity heuristics. Arianne also did the documentation for the syntaxes of code she did and provided the screenshots of it.

Lester on the other hand integrated the Tkinter GUI layout and engineered the string adaptation logic for the legal arguments. He also did the documentation for the syntaxes of code he contributed and provided the screenshots of the program's output.

A primary challenge encountered in this activity involves the alignment of the Tkinter grid constraints especially in ensuring the input table and execution text box remained responsive and readable when the window was resized by the user. Arianne and Lester also had to iteratively test and calibrate the feature weights to ensure the algorithm prioritized the transfer medium and offense type over purely monetary differences during the retrieval phase.

Overall, contriving this project demonstrated how CBR allows intelligent decision support system in terms of bypassing rigid, hardcoded rules by dynamically applying and retaining experiential knowledge from past precedents to solve scenarios.